AI/ML ENGINEER — COMPLETE FREE ROADMAP

A practical, job-ready learning path: fundamentals → machine learning → deep learning → computer vision → GenAI → deployment → MLOps → Edge AI

How to Use This Roadmap

Do not try to learn everything at once. Learn each stage, build a project, publish it on GitHub, explain what you built, and then move forward. The goal is not certificates; the goal is demonstrable engineering ability.

1. Python & Programming Foundations

Python syntax; variables and data types; functions; OOP; exceptions; modules/packages; file handling; virtual environments; debugging; basic algorithms and data structures; Git and GitHub.

Outcome: write clean Python without depending on tutorials.

2. Mathematics & Statistics

Linear algebra: vectors, matrices, matrix operations; probability; distributions; mean/variance; conditional probability; statistics; correlation; hypothesis basics; derivatives; gradients; optimization basics.

Outcome: understand what ML algorithms are doing mathematically.

3. Data Handling & SQL

NumPy; Pandas; Matplotlib; data cleaning; missing values; outliers; EDA; preprocessing; encoding; scaling; feature engineering; train/validation/test split; SQL basics: SELECT, JOIN, GROUP BY, subqueries.

Outcome: take raw data and prepare it correctly for ML.

4. Machine Learning

Supervised learning; regression; classification; logistic regression; KNN; SVM; decision trees; random forests; gradient boosting/XGBoost; Naive Bayes; clustering; PCA; anomaly detection; pipelines.

Also master cross-validation, hyperparameter tuning, precision, recall, F1, ROC-AUC, confusion matrix, overfitting/underfitting and data leakage.

5. Deep Learning

Neural networks; tensors; forward pass; loss functions; backpropagation; optimizers; regularization; batch normalization; CNNs; RNN/LSTM basics; attention; transformers; transfer learning.

Primary framework: PyTorch. Learn TensorFlow/Keras only as a secondary framework if needed.

6. Computer Vision

Image processing; OpenCV; image classification; object detection; YOLO; segmentation; tracking; data augmentation; annotation; evaluation with IoU/mAP; vision transformers; real-time inference.

Recommended specialization for an AI + embedded profile.

7. NLP & Generative AI

Text preprocessing; embeddings; transformers; LLM fundamentals; tokenization; prompting; Hugging Face; fine-tuning concepts; LoRA/PEFT basics; evaluation; safety and hallucination awareness.

Do not make prompt engineering the foundation; learn ML/DL fundamentals first.

8. RAG & AI Applications

Embeddings; chunking; retrieval; vector databases; reranking; context construction; RAG evaluation; tool/function calling; structured outputs; agents; agent workflows.

Build useful applications rather than only chatbot demos.

9. Software Engineering for AI

Linux; command line; Git/GitHub; APIs; FastAPI; REST; JSON; testing basics; logging; configuration; environment variables; Docker; basic system design.

Outcome: turn a notebook into a usable application.

10. Model Deployment

Package models; create inference APIs; Dockerize; CPU/GPU inference; ONNX; model optimization; batching; latency; memory usage; cloud basics; monitoring.

Outcome: deploy at least one model end-to-end.

11. MLOps & LLMOps

Experiment tracking; model versioning; data/model pipelines; MLflow; reproducibility; CI/CD basics; model registry concepts; monitoring; drift; LLM tracing/evaluation.

MLflow provides workflows for model training, deep learning and LLM/agent applications.

12. Edge AI

Raspberry Pi; NVIDIA Jetson; TensorRT; ONNX Runtime; quantization; pruning concepts; model compression; camera pipelines; real-time inference; hardware acceleration.

Strong specialization if you want AI + embedded/robotics roles.

13. Projects & Portfolio

Build 2 classical ML projects; 2 deep learning projects; 2 computer vision projects; 1 RAG/GenAI project; 1 deployed project; 1 end-to-end production-style project.

Every major project should have README, architecture diagram, setup steps, dataset/source, metrics, screenshots/demo, limitations and future work.

14. Interview Preparation

Python coding; SQL; ML theory; statistics; model selection; metrics; deep learning; CNN/transformers; computer vision; GenAI/RAG; deployment; system design; project explanation.

Be able to explain every project: problem → data → approach → model → training → evaluation → deployment → challenges → improvements.

Recommended Order for a Computer Vision + Edge AI Career

Python → Math/Statistics → NumPy/Pandas/SQL → Machine Learning → PyTorch/Deep Learning → Computer Vision → YOLO → GenAI/RAG → FastAPI/Docker → Deployment → MLOps → TensorRT/ONNX → Edge AI

Project Ladder

Level	Project
Beginner ML	House Price Prediction / Student Performance Prediction
Intermediate ML	Customer Churn / Fraud Detection / Disease-risk classification using a public dataset
Deep Learning	Image Classification with PyTorch
Computer Vision	YOLO object detection + custom dataset
Real-world Vision	Areca Nut Quality Detection / Weed Detection
GenAI	RAG assistant over a collection of documents
Deployment	FastAPI + Docker model-serving application
Edge AI	Real-time object detection on Raspberry Pi or Jetson
Capstone	End-to-end AI vision system: camera → preprocessing → model → optimized inference → API/UI → logging

Free Learning Resources

• Python official documentation/tutorials

• scikit-learn User Guide — algorithms, model selection, preprocessing, pipelines and common pitfalls

• PyTorch Tutorials — tensors, datasets, neural networks, autograd, optimization and model saving

• Hugging Face Learn — modern NLP/LLM resources

• Google Colab — browser-based notebooks for learning and experiments

• MLflow documentation — experiment tracking, model workflows and LLM/agent tracing

What NOT to Do

• Do not collect certificates without building projects.

• Do not jump directly into AI agents before understanding ML/DL fundamentals.

• Do not copy GitHub projects without understanding and modifying them.

• Do not train models without learning proper evaluation and data leakage prevention.

• Do not keep everything inside Jupyter notebooks; learn deployment and software engineering.

• Do not learn every framework. Pick one primary deep-learning framework and become strong with it.

Job-Ready Checklist

■ Python ■ SQL ■ Statistics ■ ML ■ PyTorch ■ Computer Vision ■ YOLO ■ GenAI/RAG ■ Git/GitHub ■ Linux ■ FastAPI ■ Docker ■ Deployment ■ MLOps ■ Edge AI ■ 8–10 strong projects ■ Can explain projects clearly

Final Principle

Learn → **Practice** → **Build** → **Deploy** → **Document** → **Explain** → **Repeat.**
You do not need to master every AI topic before applying for jobs. Become strong in the fundamentals, specialize in Computer Vision/Edge AI, and keep expanding your stack through real projects.